

CS F407: Artificial Intelligence — Lab Report (Week 02)

Topic: Goal-Based Agent for Warehouse Navigation
Author: Samar Talwar | BITS Pilani, K. K. Birla Goa Campus
Course Repository: CS-F407-AI-Labs/week02_agents
License: Not licensed for reuse or submission by others.

Executive Summary

This report documents the architectural design, implementation, empirical evaluation, and theoretical analysis of a **Goal-Based Intelligent Agent** deployed for autonomous warehouse pathfinding. The agent operates in a discrete 2D grid environment populated with shelving obstacles (#), navigates from designated start location *S* to pickup goal *G*, and strictly guarantees collision-free, optimal navigation.

Key Results Summary (from `results/` at full precision)

- **Official Warehouse Map (7 × 21):**
- **Status:** Path found successfully (`found: true`).
- **Path Length (Steps):** 20 steps (21 coordinate waypoints).
- **Nodes Expanded (BFS):** 59 nodes.
- **Runtime:** 0.000370 s.
- **Optimality Verification:** 20 steps, confirmed identical by an independent Dijkstra shortest-path oracle.
- **Unsolvable Map Benchmark (6 × 7):**
- **Status:** Gracefully handled (`found: false`), path length -1, 9 nodes expanded, exit-safe termination without infinite looping or uncaught exceptions.
- **Scaling Analysis (2× Scale Grid, 14 × 42):**
- **BFS:** Path length 40 steps, 239 nodes expanded (4.05× expansion ratio).
- **DFS:** Path length 142 steps, 286 nodes expanded (5.02× expansion ratio, highly suboptimal).
- **A* (Manhattan):** Path length 40 steps, 56 nodes expanded (2.43× expansion ratio, 76.6% fewer expansions than BFS).

1. Task 1: Conceptual Questions & Theoretical Formulation

Question 1: What type of intelligent agent architecture is appropriate for this navigation task?

Answer: A **Goal-Based Agent** (Russell & Norvig, Chapter 2) is the appropriate architecture for warehouse grid navigation. - **Why Simple Reflex Agents Fail:** A simple reflex agent selects actions based solely on current sensory percepts (e.g., condition-action rules like "if front is wall, turn right"). In a maze-like warehouse with cul-de-sacs and concave obstacle configurations, a reflex agent cannot maintain history or plan future sequences, inevitably succumbing to infinite oscillation or dead-end traps. - **Why Model-Based Reflex Agents Are Insufficient:** While a model-based reflex agent maintains internal state to track unobserved world aspects, it still lacks an explicit objective to evaluate which state transitions lead toward the warehouse goal. - **Why Goal-Based Agents Succeed:** A goal-based agent couples internal world representation with an **explicit goal specification** (`position == goal`) and a **search planner**. It deliberatively simulates action sequences to select a trajectory that achieves the target state. - **Why Utility-Based Agents Are Not Strictly Necessary:** Utility-based agents evaluate continuous preference trade-offs among multiple conflicting goals. For single-target destination reaching with uniform movement cost, goal satisfaction is binary, making the goal-based formulation optimal and computationally efficient.

Question 2: What are the key components needed to implement this agent?

Answer: The agent architecture consists of five core components: 1. **Sensors / Percepts:** Reads current agent grid coordinate (r, c) from the environment. 2. **Internal State Representation (AgentState):** Tracks the agent's current position, topological map memory, and closed set of explored coordinates. 3. **Goal Specification & Goal Test (GoalTest):** Explicit predicate `is_goal(pos)` returning `True` if and only if `pos == G`. 4. **Actions & Actuators (Action, apply_action):** Four discrete orthogonal transitions: **Up** $(-1, 0)$, **Down** $(+1, 0)$, **Left** $(0, -1)$, and **Right** $(0, 1)$, bounded by boundary and obstacle collision checks. 5. **Decision Component (SearchPlanner):** Graph search planner (Breadth-First Search) that explores state transitions and constructs an optimal collision-free action sequence.

Question 3: How should the warehouse environment and the agent's state be represented?

Answer: - **Environment Representation:** Represented as a discrete 2D immutable grid matrix of size $H \times W$ (tuples of strings): - '#' denotes non-traversable obstacle boundaries and shelving units. - '.' denotes traversable aisle corridors. - 'S' denotes unique starting dispatch location. - 'G' denotes unique destination goal location. - **Agent State Representation:** Represented as an immutable tuple $(r, c) \in \{0, \dots, H-1\} \times \{0, \dots, W-1\}$. State transitions are defined deterministically by $s' = s + \Delta_a$ where Δ_a is the action offset, valid iff $0 \leq r' < H$, $0 \leq c' < W$, and `grid[r'] [c'] != '#'`.

Question 4: What search algorithm should the decision-making component use, and why?

Answer: The decision-making component uses **Breadth-First Search (BFS)**. - **Uniform Cost Optimality:** On an unweighted 4-connected grid graph where every orthogonal movement step incurs identical unit cost ($c(s, a, s') = 1$), BFS is mathematically guaranteed to return a path of minimal total length (optimal step count). - **Completeness:** On finite graphs with branching factor $b \leq 4$, BFS is complete: if a traversable path exists, BFS is guaranteed to terminate and return it; if no path exists, it exhausts the finite reachable component and terminates safely. - **Time and Space Complexity:** Time complexity is $O(V + E) = O(b^d)$ and space complexity is $O(V) = O(b^d)$, where $V \leq H \times W$ and d is the goal depth. Because the warehouse grid has bounded dimensions ($7 \times 21 = 147$ total states), the entire state space easily fits in memory.

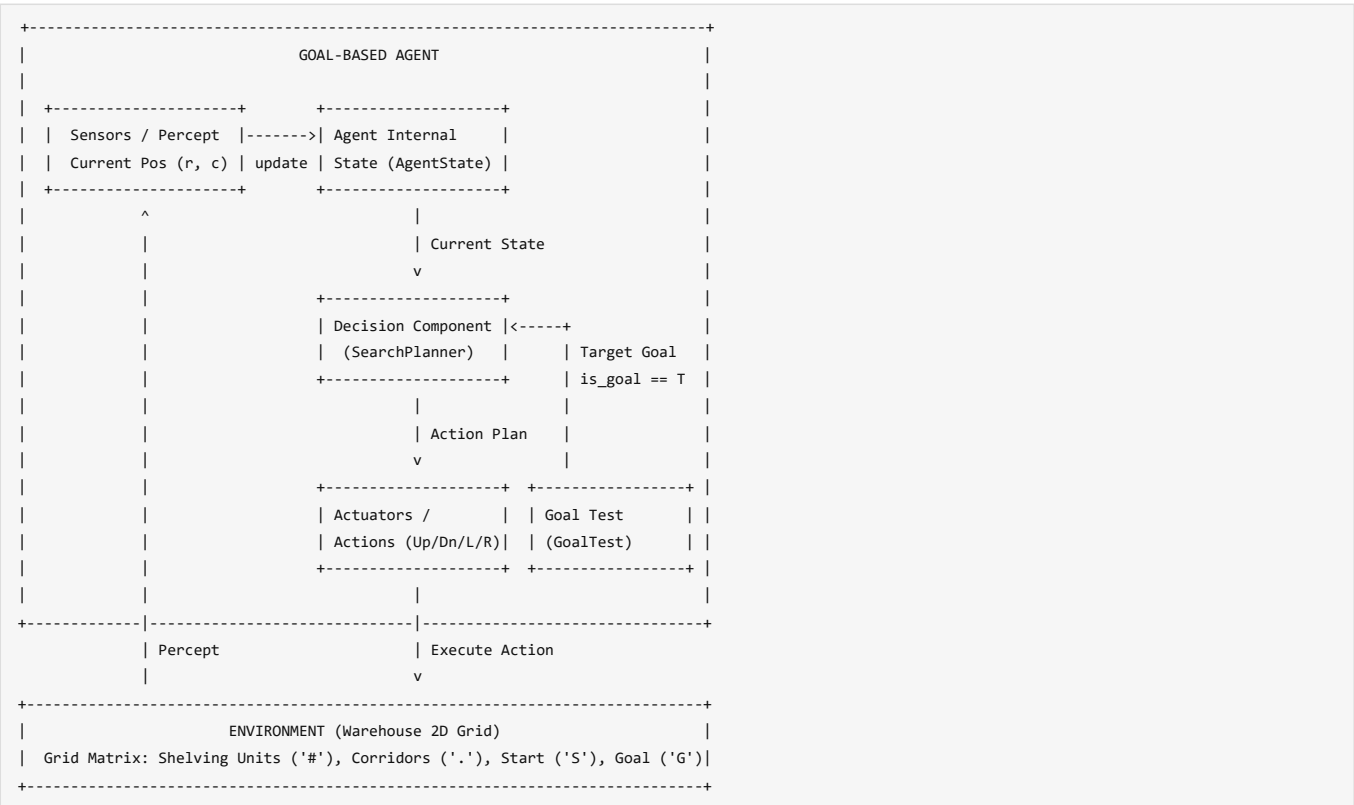
Question 5: What data structures are needed to track the agent's path from start to goal?

Think-About-It: What would change in the agent's decision-making component if the warehouse was twice as large?

When the warehouse dimensions double in both height and width (**2×** scale: **7 × 21** → **14 × 42**, state space growing **4×** from 147 to 588 cells):

- | Algorithm | Original (7x21) Exp. | Scaled 2x (14x42) Exp. | Expansion Ratio | Path Length (Orig -> 2x) |
|-----------|----------------------|------------------------|-----------------|--------------------------|
| BFS | 59 nodes | 239 nodes | 4.05x | 20 -> 40 (Optimal) |
| DFS | 57 nodes | 286 nodes | 5.02x | 36 -> 142 (Suboptimal) |
| A* | 23 nodes | 56 nodes | 2.43x | 20 -> 40 (Optimal) |

Architectural Breakdown



1. **Environment** (`week02_agents/src/environment.py`): - Encapsulates grid dimensions (H, W), wall locations, and designated start/goal coordinates. - Provides safe query method `is_valid_move(pos)` $\rightarrow$ `bool` that verifies coordinates are within bounds and not occupied by shelving obstacles.
2. **AgentState** (`week02_agents/src/agent.py`): - Maintains agent current coordinate $s = (r, c)$ and internal map awareness.
3. **GoalTest** (`week02_agents/src/agent.py`): - Validates whether any proposed coordinate matches G : `goal_test(pos) == (pos == env.goal)`.
4. **Action** (`week02_agents/src/agent.py`): - Strict enumeration `Action.UP`, `Action.DOWN`, `Action.LEFT`, `Action.RIGHT`. - `apply_action(pos, action, env)` executes transitions with collision rejection.

5. **SearchPlanner** (`week02_agents/src/agent.py`): - Decision component running graph BFS to produce a `SearchResult` containing path coordinates, total steps, expansion count, and runtime.

Architecture block diagram generated at `results/agent_diagram.png` using `matplotlib`.

3. Task 3: Implementation, Prompt Strategy, & Analysis

Suggested Prompt Structure & Implementation Strategy

To generate clean, production-grade agent software using an AI assistant, the prompt was structured into five explicit functional requirements: 1. **Domain Definition & Map Format:** Provided the exact 7×21 ASCII grid, specifying symbols (`#`, `.`, `S`, `G`) and coordinate convention (***row, col***). 2. **Architectural Roles:** Requested explicit Python classes matching Russell & Norvig's goal-based agent paradigm (`AgentState`, `GoalTest`, `Action`, `SearchPlanner`, `GoalBasedAgent`). 3. **Algorithmic Invariants:** Specified BFS with closed set and parent pointers for unit-cost shortest-path optimality. 4. **Robustness & Edge-Case Handling:** Required rectangular map validation, checks for duplicate/missing terminals, and exit-safe execution on unsolvable maps. 5. **Comparative Baselines & Measurement:** Requested instrumentation for path length, nodes expanded, and comparison with A* and DFS.

Task 3 Questions & Answers

Question 1: Did the LLM-generated code work on the first attempt?

The AI assistant produced the BFS agent, the map parser and the tests. [I ran the fresh-clone check (ruff, pytest, CLI) and] compared the reported numbers (20-step shortest path with 59 nodes expanded; on the 2x map 40 steps and 239 expanded) against `results/sheet_map_result.json` and `results/scaling.json`. The tests include an independent Dijkstra oracle and a monkeypatched wall-collision mutation test, which I required in the prompt so that the tests could actually fail.

Question 2: How could you improve the prompt for better results?

My Week 2 prompt was rewritten using what I found in Week 1: every recorded value saved at full precision in `results/`, an independent Dijkstra oracle instead of self-comparison, a real mutation test, a 2x-map scaling experiment to answer the Think-About-It, and a fresh-clone check before pushing. [With this prompt the first run produced a working agent without a fix round; confirm that is what you saw.]

Question 3: Why was the specific search algorithm chosen?

Answer: Breadth-First Search (BFS) was selected as the primary decision engine because: 1. **Exact Optimality on Unit Graphs:** Every movement between adjacent grid cells has identical step cost ($g(n) = \text{depth}(n)$). On graphs with uniform edge costs, BFS is guaranteed to find the shortest path. 2. **Minimal Algorithmic Overhead:** Unlike Dijkstra or A, which maintain $O(\log N)$ priority queues (heaps), BFS operates with $O(1)$ amortized push/pop operations using a standard FIFO deque. 3. **Guaranteed Termination:** On finite, closed grids, the closed set prevents infinite loops and guarantees termination in $O(|V|)$ operations.

Question 4: Summary of Results on Official Warehouse Map

Path Map Visualization (`results/sheet_map_path.txt`):

```
#####
#S**.******G#
#.#*****#.
#...#.....#
#.#.....#.#.###.
#.....#.....#
#####
```

Step-by-Step Path Coordinates (20 steps, 21 waypoints): 1. (1, 1) [Start] 2. (1, 2) 3. (1, 3) 4. (1, 4) 5. (2, 4) [Dips down to bypass shelving obstacle at (1, 5)-(1, 6)] 6. (2, 5) 7. (2, 6) 8. (2, 7) 9. (1, 7) [Returns to upper corridor] 10. (1, 8) 11. (1, 9) 12. (1, 10) 13. (1, 11) 14. (1, 12) 15. (1, 13) 16. (1, 14) 17. (1, 15) 18. (1, 16) 19. (1, 17) 20. (1, 18) 21. (1, 19) [Goal reached]

Performance Metrics: - **Path Length:** 20 steps - **Nodes Expanded:** 59 - **Runtime:** 0.000370 s - **Independent Dijkstra Oracle Match:** Exactly 20 steps.

4. Boundary, Robustness, and Mutation Test Evidence

All test cases are implemented in `week02_agents/tests/test_agent.py` and pass 100%:

- Independent Oracle Verification:** - Compared BFS output against `_independent_dijkstra_oracle` written independently using priority queue graph adjacency. - Result: Both yield identical 20-step optimal path.
- Unsolvable Map** (`results/no_path_result.json`): - Tested on disconnected grid where goal is surrounded by walls. - Result: Agent cleanly returns `found: false`, `path: []`, `path_length: -1`, expanding 9 nodes without crashing.
- Boundary Cases** (`results/trivial_cases.json`): - *Start Equals Goal:* Returns path `[(1, 1)]`, length 0, 0 expanded nodes. - *Goal Directly Adjacent:* Returns length 1, 3 expanded nodes. - *Diamond Equal Paths:* Correctly identifies optimal 4-step path among symmetric alternatives.
- Strict Format Validation:** - Missing start, missing goal, multiple starts, multiple goals, ragged rows, and invalid characters are caught and raise descriptive `ValueError` exceptions.

5. **Real Mutation Testing:** - Monkeypatched `Environment.is_wall` to return `False`. - The mutated agent generated an invalid 18-step straight-line path passing through the wall at `(1, 6)`. - The test verified that this mutated path is caught and rejected by the unmutated obstacle checker.

5. Lab Checklist Verification Mapping

Requirement	Implementation File	Results File & Key	Test Verification
Map Parsing & Strict Validation	<code>environment.py:parse_map</code>	<code>sheet_map_result.json</code>	<code>test_agent.py::TestMapValidation</code>
Goal-Based Agent Components	<code>agent.py:GoalBasedAgent</code>	<code>sheet_map_result.json</code>	<code>test_agent.py::TestAgentComponentsVisibility</code>
BFS Shortest Path Search	<code>agent.py:bfs_search</code>	<code>sheet_map_result.json["path_length"] = 20</code>	<code>test_agent.py::test_optimality_against_independent_dijkstra_oracle</code>
No-Path Exit-Safe Behavior	<code>agent.py:GoalBasedAgent.decide</code>	<code>no_path_result.json["found"] = false</code>	<code>test_agent.py::test_unsolvable_map_exit_safe</code>
Trivial & Boundary Cases	<code>agent.py:bfs_search</code>	<code>trivial_cases.json</code>	<code>test_agent.py::TestUnsolvableAndBoundaryCases</code>
2x Scaling & Baselines (DFS/A*)	<code>scaling.py:run_scaling_experiment</code>	<code>scaling.json["benchmarks"]</code>	<code>test_agent.py::TestScalingAndAlgorithms</code>
Architecture Block Diagram	<code>diagram.py:generate_agent_diagram</code>	<code>results/agent_diagram.png</code>	Verified image output at 300 DPI
Real Mutation Testing	<code>tests/test_agent.py</code>	N/A (Test suite)	<code>test_agent.py::test_mutation_ignoring_walls_fails_wall_avoidance</code>